East West University

Department of Computer Science and Engineering

PROJECT REPORT
Group - 3

Project Name: Postfix and Prefix Calculator

Course Code: CSE207

Course Title: Data Structures

Section: 07

Submitted by:

- Tanvir Ahmed Ratul 2021-3-60-140
- Md. Asifuzzaman 2021-3-60-167
- Fatema Tahsin Anamika 2021-3-60-170
- Halima Sadia 2021-3-60-280
- Shanta Singha 2021-3-60-298

Course Instructor: Md. Manowarul Islam

Adjunct Faculty,

Department of Computer Science & Engineering

East West University.

Introduction:

In computer science, arithmetic expressions are often represented in different notations, such as infix, prefix, and postfix. Converting expressions from one form to another is a common operation in compilers, interpreters, and mathematical software.

- **Infix Notation:** This is the standard mathematical notation where operators are written between their operands. For example, "a + b" or "(a * (b + c))".

- **Prefix Notation:** In this notation, the operator precedes their operands. For example, "+ a b" or "* a + b c".

- **Postfix Notation:** Also known as Reverse Polish Notation (RPN), operators come after their operands. For example, "a b +" or "a b c + *".

The primary purpose of the Infix to Prefix and Postfix Calculator is to convert arithmetic expressions from infix notation to prefix and postfix notations. This can be useful in various applications –

- Compiler design,
- Expression evaluation,
- Mathematical software development.

Code Structure:

- Two structures are used in this project: Stack and Queue. They use dynamic memory to store elements in arrays.
- For Stack operations, we used:
  - createStack (int capacity)
  - isEmpty (Stack* stack)
  - push (Stack* stack, char item)
  - pop (Stack* stack)
  - peek (Stack* stack)
- For Queue operations, we used:
  - createQueue (int capacity)
  - isQueueEmpty (Queue* queue)
  - enqueue (Queue* queue, char item)
  - Dequeue (Queue* queue)

- The program converts infix expressions to postfix using a stack and evaluates postfix expressions. It also reverses strings and converts infix expressions to prefix using a stack.

- The main function creates a user interface. The calculator menu shows here. The program lets the user choose to convert infix to prefix, infix to postfix or exit. It takes user input for an infix expression, performs the chosen operation, and prints the result. If the choice is invalid, it shows a message. The code runs until the user choose to exit.

The program provides a practical illustration of fundamental stack and queue operations, showcasing their utility in handling infix to postfix and infix to prefix conversions. The calculator offers users the flexibility to choose the desired conversion method and subsequently evaluates the result.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

typedef struct {
    char* array;
    int top;
    int capacity;
} Stack;

typedef struct {
    char* array;
    int front;
    int rear;
    int capacity;
} Queue;

Stack* createStack(int capacity) {
    Stack* stack = (Stack*)malloc(sizeof(Stack));
    stack->capacity = capacity;
    stack->top = -1;
    stack->array = (char*)malloc(stack->capacity * sizeof(char));
    return stack;
}

int isEmpty(Stack* stack) {
    return stack->top == -1;
```

```
}

void push(Stack* stack, char item) {
    stack->array[++stack->top] = item;
}

char pop(Stack* stack) {
    if (!isEmpty(stack))
        return stack->array[stack->top--];
    return '$';
}

char peek(Stack* stack) {
    return stack->array[stack->top];
}

Queue* createQueue(int capacity) {
    Queue* queue = (Queue*)malloc(sizeof(Queue));
    queue->capacity = capacity;
    queue->front = queue->rear = -1;
    queue->array = (char*)malloc(queue->capacity * sizeof(char));
    return queue;
}

int isQueueEmpty(Queue* queue) {
    return queue->front == -1;
}
```

```
void enqueue(Queue* queue, char item) {
    if (isQueueEmpty(queue))
        queue->front = 0;
    queue->rear++;
    queue->array[queue->rear] = item;
}

char dequeue(Queue* queue) {
    char item;
    if (!isQueueEmpty(queue)) {
        item = queue->array[queue->front];
        queue->front++;
        if (queue->front > queue->rear)
            queue->front = queue->rear = -1;
        return item;
    }
    return '$';
}

int isOperand(char ch) {
    return isalnum(ch);
}

int getPrecedence(char operator) {
    switch (operator) {
        case '+':
        case '-':
            return 1;
```

```
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        default:
            return -1;
    }
}

void infixToPostfix(char* infix, char* postfix) {
    Stack* stack = createStack(strlen(infix));
    int i, j;
    i = j = 0;

    while (infix[i] != '\0') {
        if (isOperand(infix[i]))
            postfix[j++] = infix[i++];
        else if (infix[i] == '(')
            push(stack, infix[i++]);
        else if (infix[i] == ')') {
            while (!isEmpty(stack) && peek(stack) != '(')
                postfix[j++] = pop(stack);
            pop(stack);
            i++;
        } else {
            while (!isEmpty(stack) && getPrecedence(infix[i]) <= getPrecedence(peek(stack)))
                postfix[j++] = pop(stack);
```

```
            push(stack, infix[i++]);
        }
    }

    while (!isEmpty(stack))
        postfix[j++] = pop(stack);

    postfix[j] = '\0';
}

int evaluatePostfix(char* postfix) {
    Stack* stack = createStack(strlen(postfix));
    int i, operand1, operand2, result;

    for (i = 0; postfix[i] != '\0'; ++i) {
        if (isOperand(postfix[i]))
            push(stack, postfix[i] - '0');
        else {
            operand2 = pop(stack);
            operand1 = pop(stack);
            switch (postfix[i]) {
                case '+':
                    push(stack, operand1 + operand2);
                    break;
                case '-':
                    push(stack, operand1 - operand2);
                    break;
                case '*':
```

```
                push(stack, operand1 * operand2);
                break;
            case '/':
                push(stack, operand1 / operand2);
                break;
            case '^':
                push(stack, operand1 ^ operand2);
                break;
        }
    }
}
result = pop(stack);
return result;
}

void reverseString(char* str) {
    int length = strlen(str);
    int i, j;
    char temp;
    for (i = 0, j = length - 1; i < j; ++i, --j) {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
    }
}

void infixToPrefix(char* infix, char* prefix) {
    reverseString(infix);
```

```
    Stack* stack = createStack(strlen(infix));
    int i, j;
    i = j = 0;

    while (infix[i] != '\0') {
        if (isOperand(infix[i]))
            prefix[j++] = infix[i++];
        else if (infix[i] == ')')
            push(stack, infix[i++]);
        else if (infix[i] == '(') {
            while (!isEmpty(stack) && peek(stack) != ')')
                prefix[j++] = pop(stack);
            pop(stack);
            i++;
        } else {
            while (!isEmpty(stack) && getPrecedence(infix[i]) < getPrecedence(peek(stack)))
                prefix[j++] = pop(stack);
            push(stack, infix[i++]);
        }
    }

    while (!isEmpty(stack))
        prefix[j++] = pop(stack);

    prefix[j] = '\0';
    reverseString(prefix);
}
```

```
int evaluatePrefix(char* prefix) {
    Stack* stack = createStack(strlen(prefix));
    int i, operand, result;

    for (i = strlen(prefix) - 1; i >= 0; --i) {
        if (isOperand(prefix[i]))
            push(stack, prefix[i] - '0');
        else {
            int operand1 = pop(stack);
            int operand2 = pop(stack);
            switch (prefix[i]) {
                case '+':
                    operand = operand1 + operand2;
                    break;
                case '-':
                    operand = operand1 - operand2;
                    break;
                case '*':
                    operand = operand1 * operand2;
                    break;
                case '/':
                    operand = operand1 / operand2;
                    break;
                case '^':
                    operand = operand1 ^ operand2;
                    break;
            }
```

```
            push(stack, operand);
        }
    }
    result = pop(stack);
    return result;
}

int main() {
    int choice;
    char infix[100], postfix[100], prefix[100];

    printf("=====================================================\n");
    printf("||--- Welcome to PREFIX and POSTFIX Calculator ---||\n");

    for (int i = 0; ; i++) {
        printf("=====================================================\n");
        printf("||  Choose one from the below options            ||\n");
        printf("||  1. Convert Infix to Prefix and Evaluate      ||\n");
        printf("||  2. Convert Infix to Postfix and Evaluate     ||\n");
        printf("||  3. Exit                                      ||\n");
        printf("=====================================================\n");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            printf("Enter an infix expression: ");
            scanf("%s", infix);
```

```
        infixToPrefix(infix, prefix);
        printf("Prefix expression: %s\n", prefix);

        int prefixResult = evaluatePrefix(prefix);
        printf("Result of prefix expression: %d\n\n", prefixResult);
    } else if (choice == 2) {
        printf("Enter an infix expression: ");
        scanf("%s", infix);

        infixToPostfix(infix, postfix);
        printf("Postfix expression: %s\n", postfix);

        int postfixResult = evaluatePostfix(postfix);
        printf("Result of postfix expression: %d\n\n", postfixResult);
    } else if (choice == 3) {
        printf("Exiting the program...");
        break;
    } else {
        printf("\nInvalid choice. Please enter a valid option.\n\n");
    }
  }
  return 0;
}
```

Output:

```
=====================================================
||--- Welcome to PREFIX and POSTFIX Calculator ---||
=====================================================
||  Choose one from the below options              ||
||  1. Convert Infix to Prefix and Evaluate        ||
||  2. Convert Infix to Postfix and Evaluate       ||
||  3. Exit                                        ||
=====================================================

Enter your choice: 1
Enter an infix expression: ((4+5)*2-(6-4))/(7+1)
Prefix expression: /-*+452-64+71
Result of prefix expression: 2

=====================================================
||  Choose one from the below options              ||
||  1. Convert Infix to Prefix and Evaluate        ||
||  2. Convert Infix to Postfix and Evaluate       ||
||  3. Exit                                        ||
=====================================================

Enter your choice: 2
Enter an infix expression: ((4+5)/3+(6-4))*(7+1)
Postfix expression: 45+3/64-+71+*
Result of postfix expression: 40

=====================================================
||  Choose one from the below options              ||
||  1. Convert Infix to Prefix and Evaluate        ||
||  2. Convert Infix to Postfix and Evaluate       ||
||  3. Exit                                        ||
=====================================================

Enter your choice: 4

Invalid choice. Please enter a valid option.

=====================================================
||  Choose one from the below options              ||
||  1. Convert Infix to Prefix and Evaluate        ||
||  2. Convert Infix to Postfix and Evaluate       ||
||  3. Exit                                        ||
=====================================================

Enter your choice: 3
Exiting the program...
Process returned 0 (0x0)   execution time : 265.613 s
Press any key to continue.
```